Langfuse en DocuSearch AI — investigación e integración

Fecha de investigación: 05/11/2026
Versión revisada: Langfuse v4 (self-host y Cloud) · Python SDK v5
Estado de DocuSearch AI: agente activo con **LangChain · OpenCode 2 · deepseek/deepseek-flash** y observabilidad propia en SQL Server.

Implementación en este repositorio: integración **opcional** ya incluida. Se activa únicamente si existen las tres variables `LANGFUSE_PUBLIC_KEY`, `LANGFUSE_SECRET_KEY` y `LANGFUSE_BASE_URL`; sin ellas el worker no inicializa Langfuse ni envía nada.

Archivos modificados:

- `LangChain/opencode_worker.py`: observación `generation` con solo metadatos.
- `LangChain/requirements.txt`: `langfuse>=5,<6`.
- `Services/LangChainRunner.cs`: propaga `LANGFUSE_*` al proceso Python.
- `obj/app-start.sh`: instala el SDK si hay claves configuradas.
- `obj/_docker-run.ps1`: pasa las claves del host al contenedor.

---

1. Resumen ejecutivo

Langfuse es una plataforma open source de observabilidad para aplicaciones LLM: captura trazas, generaciones, prompts, tokens, latencia, errores, sesiones y evaluaciones. Puede usarse en la nube o autoalojada.

¿Se puede integrar? Sí, y la vía más natural es instrumentar el **worker Python de LangChain** que ya usa DocuSearch AI. Hay dos caminos:

1. **Rápido:** Langfuse Cloud (gratis para probar), útil solo con datos sintéticos.
2. **Alineado con local-first:** Langfuse autoalojado con Docker Compose, en la misma máquina o en un servidor interno.

Recomendación: no agregarlo al demo por defecto. Integrarlo como **observabilidad opcional de desarrollo**, con **solo metadatos** y autoalojado. Langfuse captura por defecto prompts y respuestas, y eso choca con la promesa actual de DocuSearch AI de no guardar contenido en las trazas.

---

2. Qué es Langfuse

Capacidad	Para qué sirve
Tracing	Ver el recorrido completo de una consulta: prompt, generación, tokens, latencia, errores
Generations	Registrar modelo, proveedor, uso de tokens y coste
Sesiones y usuarios	Agrupar consultas por conversación o usuario
Prompt management	Versionar y comparar prompts
Evaluaciones	Puntuar respuestas, comparar modelos, regresiones
Dashboards	Latencia, errores, coste y uso por modelo

En **v4**, Langfuse usa una arquitectura con varios componentes:

- `langfuse-web`: UI y API.
- `langfuse-worker`: procesamiento asíncrono.
- PostgreSQL: datos transaccionales.
- ClickHouse: trazas y observaciones.
- Redis/Valkey: cola y caché.
- S3/MinIO: eventos y adjuntos.

La versión self-host por Docker Compose es para **uso local o una VM**, sin alta disponibilidad ni backups automáticos. La documentación recomienda **4 cores y 16 GiB** para una VM de producción; en un portátil con Docker Desktop puede ser

pesado.

3. Cómo encaja con DocuSearch AI

DocuSearch AI ya tiene una observabilidad propia:

- `AgentTraces` en SQL Server: motor, modelo, duración, tokens, estado y código de error.
- Página **Observabilidad** para Administración General y Auditoría.
- Auditoría `AGENT_QUERY` sin contenido.
- Sin LangSmith ni telemetría externa.

El flujo generativo actual:

```
Blazor / C#
  → LocalAgent
  → OpenCodeClient (UseLangChain=true)
  → LangChainRunner (proceso Python aislado)
  → LangChain: prompt | RunnableLambda(call_opencode) | StrOutputParser
  → OpenCode 2 (sesión con permisos deny)
  → respuesta JSON + citas
  → validación en C#
```

Langfuse puede observar ese proceso, pero hay que decidir **qué datos enviar**:

- Solo metadatos (recomendado): modelo, proveedor, duración, tokens, estado, cantidad de fuentes, error.
- Contenido completo: pregunta, fragmentos y respuesta. Útil para depurar, pero **sensible** y contradictorio con la política actual.

4. Opciones de integración

Opción	Esfuerzo	Ventaja	inconveniente
A. Langfuse Cloud	Bajo	Sin infraestructura; UI inmediata	Los datos salen de la máquina; no cumple local-first
B. Langfuse self-host	Medio	Datos locales; control total	Stack pesado (Postgres, ClickHouse, Redis, MinIO, web, worker)
C. OTel desde C#	Alto	Cubre también el modo OpenCode directo	Más código; atributos GenAI; no hay SDK oficial C# de Langfuse
D. Exportar `AgentTraces`	Medio	Reutiliza lo ya guardado	No es tiempo real; hay que mapear a la API/OTLP

Para DocuSearch AI: empezar por **B + worker LangChain**, con contenido enmascarado o solo metadatos. La opción C queda para una segunda fase si se quiere observar el modo `UseLangChain=false`.

5. Recomendación por fases

Fase 0 — No tocar el demo

Mantener Langfuse **desactivado** en la presentación. La app ya demuestra observabilidad propia y la promesa de no enviar contenido.

Fase 1 — Prototipo en desarrollo

1. Levantar Langfuse self-host en Docker.
2. Crear organización, proyecto y API keys.
3. Instrumentar el worker LangChain con **solo metadatos**.
4. Ver una consulta en la UI.
5. Verificar que si Langfuse está caído, la consulta sigue funcionando.

Fase 2 — Opcional para producción

1. Self-host en una VM interna con backups y retención.

2. Activar autenticación, TLS y red restringida.
3. Evaluar si se instrumenta el modo OpenCode directo con OTel desde C#.
4. Actualizar la documentación y la presentación: dejar de decir "sin telemetría" si Langfuse queda activo.

---

6. Self-host local paso a paso

Requisitos: Git, Docker y Docker Compose.

```
git clone https://github.com/langfuse/langfuse.git
cd langfuse
# Editar docker-compose.yml y cambiar todos los secretos marcados con # CHANGEME
docker compose up -d
```

Esperar a que `langfuse-web` esté listo (unos 2–3 minutos). Abrir:

```
http://localhost:3000
```

No hay usuario admin por defecto: pulsar **Sign up**, crear el primer usuario, la organización y el proyecto. Luego crear **API keys**:

```
pk-lf-...   → pública
sk-lf-...   → secreta
```

Conectividad desde DocuSearch AI

El worker Python corre dentro del contenedor `ds-app`. Langfuse puede estar:

- En el host, con el puerto 3000 publicado: `LANGFUSE_BASE_URL=http://host.docker.internal:3000`.
- En la misma red Docker que `ds-app`: usar el nombre del servicio, por ejemplo `http://langfuse-web:3000`.

No exponer la UI a internet sin TLS y autenticación.

---

7. Integrar Langfuse en el worker LangChain

7.1 Dependencia

El worker actual usa `langchain-core==1.7.0`. Langfuse Python SDK v5 requiere `langchain-core>=1`, por lo que es compatible.

```
# LangChain/requirements.txt
langfuse>=5,<6
```

En el contenedor:

```
docker exec ds-app pip install "langfuse>=5,<6"
```

Para que sea reproducible, agregar la dependencia a `requirements.txt` y reconstruir la imagen/instalar en `app-start.sh`.

7.2 Variables de entorno

```
LANGFUSE_PUBLIC_KEY=pk-lf-...
LANGFUSE_SECRET_KEY=sk-lf-...
LANGFUSE_BASE_URL=http://host.docker.internal:3000
```

No guardar estas claves en el repositorio. Usar variables de entorno o Docker secrets.

7.3 Problema específico de DocuSearch AI

`LangChainRunner` hace `start.Environment.Clear()` y solo conserva `SystemRoot`, `WINDIR`, `TEMP`, `TMP` y `LOCALAPPDATA`. Por eso, aunque definas `LANGFUSE_*` en el contenedor, **el worker no las recibe**.

En este repositorio ya se añadieron explícitamente en `Services/LangChainRunner.cs`:

```
foreach (var key in new[]
{
    "SystemRoot", "WINDIR", "TEMP", "TMP", "LOCALAPPDATA",
    "LANGFUSE_PUBLIC_KEY", "LANGFUSE_SECRET_KEY", "LANGFUSE_BASE_URL"
})
{
    if (Environment.GetEnvironmentVariable(key) is { } value)
        start.Environment[key] = value;
}
```

Nunca registrar esas variables en logs.

7.4 Instrumentación del worker

El worker no llama a un `ChatModel` de LangChain: llama a OpenCode con `httpx` dentro de un `RunnableLambda`. Por eso el `CallbackHandler` de LangChain vería la cadena, pero **no la generación real**. Es mejor crear una observación manual alrededor de `call_opencode`.

Bosquejo:

```
import os
from langfuse import get_client

langfuse = get_client() if os.getenv("LANGFUSE_PUBLIC_KEY") else None


def invoke_model(prompt_value):
    if langfuse is None:
        text, tokens = call_opencode(request, prompt_value)
        return text

    with langfuse.start_as_current_observation(
        as_type="generation",
        name="opencode-generation",
        model=request.model_id,
        input={  # solo metadatos, no la pregunta ni los fragmentos
            "question_length": len(request.question),
            "source_count": len(request.sources),
            "provider": request.provider_id,
        },
    ) as generation:
        text, tokens = call_opencode(request, prompt_value)
        generation.update(
            output={"answer_length": len(text)},
            metadata={
                "engine": "opencode",
                "langchain": True,
                "document_count": len(request.sources),
            },
        )
        return text
```

Al final del proceso, **flush explícito**:

```
finally:
    if langfuse is not None:
        langfuse.flush()
```

Sin `flush()`, un proceso corto como el worker puede terminar antes de enviar los eventos.

También se puede usar el callback nativo de LangChain:

```
from langfuse.langchain import CallbackHandler

handler = CallbackHandler()
build_chain(request).invoke({"context": context}, config={"callbacks": [handler]})
```

Pero, como se explicó, no capturará la llamada real a OpenCode como una generación. Para eso sirve la observación manual.

7.5 No romper la consulta

Langfuse debe ser **best-effort**:

- Inicializar solo si hay claves.
- Envolver la instrumentación en `try/except`.
- Si Langfuse no responde, la generación debe continuar.
- El `flush()` no debe bloquear indefinidamente.

7.6 Activar Langfuse en DocuSearch AI

En la misma PowerShell desde la que se lanza el contenedor:

```
$env:LANGFUSE_PUBLIC_KEY="pk-lf-..."
$env:LANGFUSE_SECRET_KEY="sk-lf-..."
$env:LANGFUSE_BASE_URL="http://host.docker.internal:3000"
& "C:\Users\Usuario\Documents\Programación\DocuSearch-AI\obj\_docker-run.ps1"
```

`_docker-run.ps1` pasa las claves al contenedor y `app-start.sh` instala el SDK si hace falta. No es necesario editar archivos ni poner las claves en Git.

Para desactivarlo, elimina esas variables y vuelve a crear el contenedor. La app seguirá funcionando igual, sin enviar trazas a Langfuse.

---

8. Instrumentar el modo OpenCode directo (opcional, C#)

Si `UseLangChain=false`, el cliente C# habla directamente con OpenCode. Para observarlo, Langfuse expone un endpoint **OTLP**:

```
http://localhost:3000/api/public/otel
Authorization: Basic <base64(public_key:secret_key)>
x-langfuse-ingestion-version: 4
```

En .NET se puede usar `OpenTelemetry.Exporter.OpenTelemetryProtocol` con HTTP/protobuf (gRPC no está soportado) y crear spans con atributos GenAI:

- `gen_ai.system = opencode`
- `gen_ai.request.model = deepseek-flash`
- `gen_ai.usage.input_tokens` / `output_tokens`
- duración, estado y error

Esto requiere más trabajo que el worker y no es necesario en la primera fase, porque el modo activo es LangChain.

---

9. Privacidad, masking y expectativas

Langfuse captura por defecto **input/output** de las observaciones. En DocuSearch AI eso puede incluir:

- la pregunta del usuario;
- fragmentos de documentos;
- la respuesta generada;
- nombres de expedientes o datos personales.

Opciones

1. **Solo metadatos (recomendado):** no enviar pregunta, fragmentos ni respuesta; solo modelo, tokens, duración, estado, cantidad de fuentes y error.
2. **Masking en el cliente:** usar el hook de masking del SDK (`mask` o `mask_otel_spans`, según versión) para redactar datos antes de enviarlos.
3. **Self-host con retención corta:** datos locales, pero igualmente persistentes; definir retención y acceso.

La documentación oficial de Langfuse describe masking en cliente y masking en ingestión (esta última requiere licencia EE).

> Importante: si Langfuse queda activo, la frase "sin telemetría externa" de la presentación deja de ser exacta. Aunque sea self-host, hay un backend de observabilidad adicional. Actualizar la documentación o mantenerlo desactivado en el demo.

10. Correlación con `AgentTraces`

DocuSearch AI ya guarda una traza en SQL Server por consulta. Para no duplicar:

- Añadir el `AgentTrace.Id` como metadata de Langfuse (`docsearch.trace_id`).
- Mantener SQL como fuente de auditoría y Langfuse como herramienta de análisis/desarrollo.
- Evitar enviar dos veces el mismo contenido; enviar metadatos.

11. Verificación

- [ ] Langfuse UI abre en `http://localhost:3000`.
- [ ] Se creó organización, proyecto y API keys.
- [ ] El worker recibe `LANGFUSE_*`.
- [ ] Una consulta aparece en Langfuse como traza.
- [ ] La traza **no** contiene pregunta, fragmentos ni respuesta (si se eligió metadata-only).
- [ ] Si Langfuse está apagado, la consulta sigue respondiendo.
- [ ] `langfuse.flush()` se ejecuta al terminar el worker.
- [ ] Las claves no están en Git.
- [ ] Se actualizó la documentación sobre telemetría.

12. Riesgos y limitaciones

Riesgo	Impacto
Stack self-host pesado	Consume CPU, RAM y disco; puede afectar la demo local
Datos sensibles	Langfuse guarda prompts/respuestas si no se enmascara
Worker de corta vida	Sin `flush()` se pierden eventos
`LangChainRunner` limpia el entorno	Las claves no llegan al worker si no se whitelistean
Modo OpenCode directo	Requiere OTel en C#, más esfuerzo
Dependencia adicional	Actualizaciones, backups, migraciones y seguridad
Expectativa de privacidad	Cambia la promesa "sin telemetría"

13. Esfuerzo estimado

Alcance	Estimación
Prototipo Langfuse Cloud + worker	2–4 horas
Self-host local + worker + masking	1 día
Instrumentación C#/OTel del modo directo	1–2 días
Producción (TLS, backups, retención, RBAC)	2–4 días

14. Conclusión

Langfuse es una buena herramienta para **desarrollar y evaluar** el agente, pero **no es necesaria para el demo actual**. Si se integra, la opción más coherente con DocuSearch AI es:

1. **Self-host** en la misma red.
2. **Solo metadatos** o masking estricto.
3. Instrumentar el **worker LangChain** primero.
4. Mantenerlo **desactivado por defecto** y actualizar las afirmaciones de privacidad si se activa.

Fuentes

- Langfuse — Self-hosting
- Langfuse — Docker Compose
- Langfuse — LangChain integration
- Langfuse — SDK overview
- Langfuse — Instrumentation
- Langfuse — OpenTelemetry
- Langfuse — Masking
- Langfuse — Data masking self-hosted